Victoria Park Academy Shenzhen

CIE IGCSE Computer Science (0478) — YEAR 10 SEMESTER 1

COURSE OVERVIEW

1. COURSE SUMMARY

This semester has a dual purpose:

(a) NEW CONTENT — This curriculum systematically fills the five remaining syllabus gaps that were not covered in Year 9, ensuring every CIE 0478 specification point has been taught at least once before the final examination year.

(b) CONSOLIDATION — The programme includes a structured, spiral review of ALL Year 9 topics (Data Representation, Communication, Hardware, Software, Security & Ethics, Algorithm Design, Programming Fundamentals) so that by the end of Year 10 students have encountered every specification topic twice with increasing depth.

By the end of Y10S1, students would have:

- Completed first exposure to all Paper 1 theory topics (Topics 1–10)
- Completed first exposure to all Paper 2 programming topics
- A personal revision resource folder organised by topic
- Experienced one full Paper 1 mock examination under timed conditions

Semester structure: 18 weeks | ~72 lessons (4 x 40-min periods per week)

2. CONNECTION TO YEAR 9

YEAR 9 SEMESTER 1 (Theory Foundations)

Topics covered: Data Representation (binary, hex, ASCII, sound, images),
Communication (Internet, WWW, email, protocols),
Hardware (CPU, memory, I/O, storage),
Software (system software, application software, utilities),
Security & Ethics (malware, hacking, backups, ethics, DPA)

→ Y10S1 Unit 5 reviews and deepens all of these.

YEAR 9 SEMESTER 2 (Programming Foundations)

Topics covered: Algorithm design (flowcharts, pseudocode, trace tables),
Programming fundamentals (variables, operators, selection,
iteration, arrays, string handling, file handling,
subroutines, structured programming)

→ Y10S1 Unit 6 reviews and deepens all of these.

THE FIVE SYLLABUS GAPS FILLED THIS SEMESTER

Gap 1 — Databases & SQL (Topic 9)

Syllabus refs: 1.3.1–1.3.4, 2.1.3–2.1.5

Content: flat-file databases vs relational databases, tables, records, fields, primary keys, foreign keys, relationships (1:1, 1:M, M:N), entity–relationship diagrams (ERDs), normalisation (1NF, 2NF, 3NF), SQL: SELECT, FROM, WHERE, ORDER BY, wildcards (% and _), INSERT, UPDATE, DELETE, JOIN operations.

Gap 2 — Boolean Logic (Topic 10)

Syllabus refs: 1.3.5–1.3.6, 2.1.6

Content: AND, OR, NOT, NAND, NOR, XOR gates; truth tables for all 6 gates; combining gates to form logic circuits; writing logic expressions from circuits and circuits from expressions; introduction to half-adders as a practical application.

Gap 3 — Bubble Sort (Topic 7)

Syllabus refs: 2.2.1, 2.2.3

Content: bubble sort algorithm step-by-step, carrying out a bubble sort by hand, writing bubble sort in pseudocode, trace tables, comparing efficiency to other sorting algorithms (briefly), Big O notation (n^2) at an introductory level.

Gap 4 — Validation Checks (Topic 8)

Syllabus refs: 2.2.2

Content: range check, type check, length check, presence check, format check; difference between validation and verification; implementing validation in Python code.

Gap 5 — Network Hardware (Topic 3.4)

Syllabus refs: 1.2.4–1.2.5, 3.4.1–3.4.4

Content: Network Interface Card (NIC), MAC address, IP address (IPv4), routers, switches, access points; network topologies (star, bus, mesh, hybrid); client-server vs peer-to-peer.

3. EIGHTEEN-WEEK CURRICULUM MAP

Week → Unit → Topic / Focus → Assessment

1 → 1 → Databases intro: tables, records, → —
fields, primary & foreign keys
2 → 1 → SQL: SELECT, WHERE, wildcards, ORDER → —
BY; practical SQLite exercises
3 → 1 → Relationships, ERDs, normalisation → Quiz 1
(1NF, 2NF, 3NF) → (Databases)
4 → 2 → Boolean Logic: AND, OR, NOT gates; → —
truth tables; logic expressions
5 → 2 → NAND, NOR, XOR; combining gates; → Quiz 2
half-adder introduction → (Boolean)
6 → 3 → Bubble sort algorithm; trace tables; → —
pseudocode; efficiency
7 → 3 → Validation checks (range, type, → Quiz 3
length, presence, format); Python code → (Sort/Valid)
8 → 4 → NIC, MAC, IP addresses; routers, → Quiz 4
switches; topologies; client-server → (Networks)
9 → 5 → Paper 1 Review: Data Representation → —
(binary, hex, ASCII, sound, images)
10 → 5 → Paper 1 Review: Communication & → —
Internet (protocols, email, WWW)
11 → 5 → Paper 1 Review: Hardware (CPU, memory, → —
storage, I/O) & Software
12 → 5 → Paper 1 Review: Security, Ethics & → —
Databases consolidation
13 → 6 → Paper 2 Review: Algorithm design & → —
pseudocode fundamentals
14 → 6 → Paper 2 Review: Programming constructs → —
(selection, iteration, arrays)
15 → 6 → Paper 2 Review: String handling & → —
file handling
16 → 6 → Paper 2 Review: Subroutines, → —
structured programming & Boolean logic
17 → 7 → MOCK EXAM — Paper 1 (1h 45m) → Mock Exam
18 → 7 → Mock exam review, target setting, → —
preparation for Y10S2

4. ASSESSMENT CALENDAR

Week 3 : Quiz 1 — Databases & SQL (20 min, 25 marks)

Week 5 : Quiz 2 — Boolean Logic (15 min, 20 marks)

Week 7 : Quiz 3 — Sorting & Validation (15 min, 20 marks)

Week 8 : Quiz 4 — Network Hardware (15 min, 20 marks)

Week 17 : MOCK EXAM — Paper 1 (1h 45m, 75 marks)

No other formal assessments this semester. All four quizzes are low-stakes retrieval practice; the mock exam is the single high-stakes assessment.

Grade boundaries for mock (CIE standard Paper 1):

A* : 67–75 A : 60–66 B : 53–59 C : 46–52 D : 39–45 E : 32–38

(Adjusted by cohort performance if necessary.)

5. STUDENT PROFILE

Cohort characteristics:

- Year 10 mixed-ability classes (typically 18–24 students per class)
- English as an Additional Language (EAL) — majority are L1 Mandarin speakers
- Prior attainment: one year of IGCSE CS (Y9S1 theory + Y9S2 programming)
- Varied programming confidence — some students fluent in Python, others still building syntactic fluency
- Strong work ethic generally; respond well to structure and visual organisers
- 40-minute lessons require tight pacing and clear routines

Implications for teaching:

- Every lesson must include a bilingual element (key term + 中文 + pinyin)
- Visual aids, diagrams, and colour-coding are essential
- Pair-programming and think-pair-share strategies are highly effective
- Retrieval starters (5 min) at the beginning of every lesson are non-negotiable — they build long-term retention for EAL learners
- Code demonstrations must be slow, narrated, and with on-screen annotations

6. STANDARD LESSON STRUCTURE (40 minutes)

[0–5] RETRIEVAL STARTER

- 3–5 short questions from last lesson / last week / last month
- Mini-whiteboards or oral response
- Instant feedback — this is the most important 5 minutes

[5–10] NEW CONTENT — INPUT

- Bilingual key term introduced with visual
- Teacher explanation / demonstration / worked example
- Students annotate printed notes or copy into books

[10–25] GUIDED PRACTICE

- Structured activity with high support (scaffolds, sentence starters, partially completed examples)

- Teacher circulates, uses hinge questions at ~15 min
- Mini-plenary to check understanding before independent work

[25–35] INDEPENDENT PRACTICE

- Students apply new knowledge with decreasing scaffolding
- Extension task ready for fast finishers
- Teacher provides 1:1 / small-group support to struggling students

[35–40] PLENARY & EXIT TICKET

- One question summarising today's learning
- Students write answer on slip / in book
- Teacher reviews after lesson to plan next lesson

7. EAL SUPPORT CHECKLIST

For every lesson, ensure the following are in place:

[ ] Bilingual glossary slide visible throughout the lesson

[ ] Key terms spoken AND written in both languages

[ ] Visual icon or diagram for every abstract concept

[ ] Sentence starters provided for written responses

[ ] Wait time of 3–5 seconds after every question

[ ] Opportunities for oral rehearsal in pairs before written work

[ ] Cognates highlighted (e.g., "database" → 数据库 shujuku; "logic" → 逻辑 luoji)

[ ] Colour-coded text for different grammatical functions

[ ] Printed notes provided so students can listen rather than copy

[ ] Differentiated questioning — directed, modified, or extended

Academic language focus by unit:

Unit 1: describing relationships, using command verbs (SELECT, WHERE)

Unit 2: explaining processes ("If A AND B are true, then...")

Unit 3: sequencing language ("first", "then", "next", "finally", "pass")

Unit 4: comparing and contrasting ("whereas", "however", "both")

Unit 5–6: exam command words ("state", "describe", "explain", "compare")

8. DIFFERENTIATION FRAMEWORK

Three-tier support model used in every lesson:

TIER 1 — CORE (All students)

- Access to printed notes, bilingual glossary, worked examples
- Retrieval starters to build automaticity
- Structured worksheets with gradual release of responsibility

TIER 2 — SUPPORT (Students needing additional help)

- Sentence starters for all written responses
- Partially completed notes / gap-fill versions
- Pre-printed pseudocode / code templates to complete
- Pairing with a supportive peer (carefully selected)
- Extra 1:1 check-in during guided practice phase
- Simplified word problems with smaller data sets

TIER 3 — EXTENSION (Students working above expected level)

- Open-ended problem-solving tasks
- Research and explain tasks (e.g., "Find out how a full adder works")
- Cross-topic challenge questions (e.g., "How could you use SQL to validate data before it is entered?")
- Peer tutoring opportunities — explain to others
- Introduction to A-Level concepts (e.g., SQL injection attacks, De Morgan's laws in Boolean algebra)

Mixed-ability strategies:

- Homogeneous pairing for coding tasks (similar ability collaborate)
- Heterogeneous grouping for discussion tasks (peer explanation)
- "Try the pink task first, then move to the green" — self-selecting difficulty with teacher guidance

9. RESOURCE REQUIREMENTS

Hardware & Software:

- Student laptops (1 per student) with Thonny IDE installed
- SQLite Browser (DB Browser for SQLite) — free, cross-platform
- Logic gate simulator — recommended: "LogicCircuit" (free, Windows) or printed logic gate templates for offline use
- Projector / interactive whiteboard for teacher demonstrations
- Mini-whiteboards and pens (1 per student) — ESSENTIAL for retrieval

Physical Materials:

- Printed A4 worksheets for every lesson (students keep in folder)
- A3 paper for ERD and topology diagram drawing
- Coloured pens / highlighters for colour-coding notes
- Revision card templates (blank, for student self-quizzing)
- Past paper packs (organised by topic) — Cambridge 0478 specimens, February/March and June series from 2020 onwards

Digital Resources:

- Class shared drive / VLE with all lesson resources organised by week
- Cambridge 0478 syllabus document (always accessible)
- Examiner reports for past papers (teacher reference)
- Mark schemes for all assessments (teacher only until after exam)

10. KEY BILINGUAL GLOSSARY — 80+ TERMS

UNIT 1 — DATABASES & SQL (数据库与 SQL)

English	Chinese (Simplified)	Pinyin
database	数据库	shujuku
table	表	biao
record	记录	jilu
field	字段	ziduan
primary key	主键	zhujian
foreign key	外键	waijian
flat-file database	平面文件数据库	pingmian wenjian shujuku
relational database	关系型数据库	guanxi xing shujuku
entity	实体	shiti
relationship	关系	guanxi
entity-relationship diagram (ERD)	实体关系图	shiti guanxi tu
cardinality	基数	jishu
one-to-one (1:1)	一对一	yi dui yi
one-to-many (1:M)	一对多	yi dui duo
many-to-many (M:N)	多对多	duo dui duo
normalisation	规范化	guifanhua
first normal form (1NF)	第一范式	di yi fanshi
second normal form (2NF)	第二范式	di er fanshi
third normal form (3NF)	第三范式	di san fanshi
redundancy	冗余	rongyu
data integrity	数据完整性	shuju wanzheng xing
Structured Query Language (SQL)	结构化查询语言	jiegouhua chaxun yuyan
query	查询	chaxun
SELECT	选择/查询	xuanze / chaxun
FROM	从…	cong
WHERE	条件/哪里	tiaojian / nali

ORDER BY	排序	paixu
ASC / DESC	升序/降序	shengxu / jiangxu
wildcard	通配符	tongpeifu
INSERT	插入	charu
UPDATE	更新	gengxin
DELETE	删除	shanchu
JOIN	连接	lianjie
INNER JOIN	内连接	nei lianjie

UNIT 2 — BOOLEAN LOGIC (布尔逻辑)

English	Chinese (Simplified)	Pinyin
Boolean	布尔	buer
Boolean algebra	布尔代数	buer daishu
logic gate	逻辑门	luoji men
AND gate	与门	yu men
OR gate	或门	huo men
NOT gate	非门	fei men
NAND gate	与非门	yu fei men
NOR gate	或非门	huo fei men
XOR gate	异或门	yi huo men
truth table	真值表	zhenzhi biao
logic expression	逻辑表达式	luoji biaodashi
logic circuit	逻辑电路	luoji dianlu
input	输入	shuru
output	输出	shuchu
Boolean variable	布尔变量	buer bianliang
truth value	真值	zhenzhi
true	真/正确	zhen / zhengque
false	假/错误	jia / cuowu
half-adder	半加器	banjia qi
full-adder	全加器	quanjia qi
De Morgan's laws	德摩根定律	De Mogen dinglu

UNIT 3 — SORTING & VALIDATION (排序与验证)

English	Chinese (Simplified)	Pinyin

bubble sort	冒泡排序	maopao paixu
sorting algorithm	排序算法	paixu suanfa
algorithm	算法	suanfa
pass	趟/轮	tang / lun
swap	交换	jiaohuan
compare	比较	bijiao
sorted	已排序	yi paixu
unsorted	未排序	wei paixu
efficiency	效率	xiaolu
Big O notation	大 O 表示法	da O biaoshifa
linear search	顺序搜索	shunxu sousuo
binary search	二分搜索	erfen sousuo
validation	验证	yanzheng
verification	校验	jiaoyan
range check	范围检查	fanwei jiancha
type check	类型检查	leixing jiancha
length check	长度检查	changdu jiancha
presence check	存在检查	cunzai jiancha
format check	格式检查	geshi jiancha
check digit	校验位	jiaoyan wei
parity check	奇偶校验	ji'ou jiaoyan

UNIT 4 — NETWORK HARDWARE (网络硬件)

English	Chinese (Simplified)	Pinyin
Network Interface Card (NIC)	网络接口卡	wangluo jiekou ka
MAC address	MAC 地址/物理地址	MAC dizhi / wuli dizhi
IP address	IP 地址	IP dizhi
IPv4	IPv4	IPv4
router	路由器	luyou qi
switch	交换机	jiaohuan ji
access point	接入点	jieru dian
network topology	网络拓扑	wangluo tuopu
star topology	星型拓扑	xing xing tuopu
bus topology	总线拓扑	zongxian tuopu

mesh topology	网状拓扑	wangzhuang tuopu
hybrid topology	混合拓扑	hunhe tuopu
client-server	客户机-服务器	kehuji-fuwuqi
peer-to-peer (P2P)	点对点	dian dui dian
packet	数据包	shuju bao
protocol	协议	xieyi
bandwidth	带宽	daikuan
latency	延迟	yanchi
Ethernet	以太网	yitai wang
Wi-Fi	无线局域网	wuxian juyu wang

YEAR 9 REVISION TERMS (Y9 复习词汇)

English	Chinese (Simplified)	Pinyin
binary	二进制	erjinzhi
hexadecimal	十六进制	shiliu jinzhi
denary / decimal	十进制	shijinzhi
ASCII	ASCII 码	ASCII ma
Unicode	Unicode 码	Unicode ma
pixel	像素	xiangsu
resolution	分辨率	fenbianlu
sampling rate	采样率	caiyanglu
sampling resolution	采样分辨率	caiyang fenbianlu
bit depth	位深度	wei shendu
compression	压缩	yasuo
lossy compression	有损压缩	yousun yasuo
lossless compression	无损压缩	wusun yasuo
Internet	互联网	hulianwang
World Wide Web (WWW)	万维网	wanweiwang
URL	统一资源定位符	tongyi ziyuan dingwei fu
HTTP / HTTPS	超文本传输协议	chaowenben chuanshu xieyi
FTP	文件传输协议	wenjian chuanshu xieyi
email protocol	电子邮件协议	dianzi youjian xieyi
IP address	IP 地址	IP dizhi
DNS	域名系统	yuming xitong
web browser	网页浏览器	wangye liulan qi
web server	网页服务器	wangye fuwuqi

HTML	超文本标记语言	chaowenben biaoji yuyan
CSS	层叠样式表	cengdie yangshi biao
JavaScript	JavaScript 脚本语言	JavaScript jiaoben yuyan
CPU	中央处理器	zhongyang chuli qi
ALU	算术逻辑单元	suanshu luoji danyuan
CU	控制单元	kongzhi danyuan
register	寄存器	jicun qi
clock speed	时钟速度	shizhong sudu
core	核心	hexin
cache	缓存	huancun
RAM	随机存取存储器	suiji cunqu cunchu qi
ROM	只读存储器	zhidu cunchu qi
virtual memory	虚拟内存	xuni neicun
secondary storage	辅助存储器	fuzhu cunchu qi
magnetic storage	磁存储	ci cunchu
optical storage	光存储	guang cunchu
solid-state storage	固态存储	gutai cunchu
input device	输入设备	shuru shebei
output device	输出设备	shuchu shebei
sensor	传感器	chuangan qi
actuator	执行器	zhixing qi
operating system	操作系统	caozuo xitong
utility software	工具软件	gongju ruanjian
compiler	编译器	bianyi qi
interpreter	解释器	jieshi qi
assembler	汇编器	huibian qi
malware	恶意软件	eyi ruanjian
virus	病毒	bingdu
phishing	网络钓鱼	wanglu diaoyu
hacking	黑客攻击	heike gongji
firewall	防火墙	fanghuo qiang
encryption	加密	jiami
decryption	解密	jiemi
private key	私钥	siyao
public key	公钥	gongyao

backup 备份 beifen

PROGRAMMING TERMS (编程术语)

English	Chinese (Simplified)	Pinyin
variable	变量	bianliang
constant	常量	changliang
assignment	赋值	fuzhi
operator	运算符	yunsuan fu
arithmetic operator	算术运算符	suanshu yunsuan fu
relational operator	关系运算符	guanxi yunsuan fu
logical operator	逻辑运算符	luoji yunsuan fu
selection	选择	xuanze
IF statement	IF 语句	IF yuju
ELSE statement	ELSE 语句	ELSE yuju
nested IF	嵌套 IF	qiantao IF
iteration	迭代/循环	diedai / xunhuan
FOR loop	FOR 循环	FOR xunhuan
WHILE loop	WHILE 循环	WHILE xunhuan
REPEAT…UNTIL	REPEAT…UNTIL 循环	REPEAT UNTIL xunhuan
array	数组	shuzu
2D array	二维数组	erwei shuzu
string	字符串	zifuchuan
string handling	字符串处理	zifuchuan chuli
concatenation	连接	lianjie
substring	子字符串	zi zifuchuan
length (of string)	长度	changdu
file handling	文件处理	wenjian chuli
read	读取	duqu
write	写入	xieru
open	打开	dakai
close	关闭	guanbi
subroutine	子程序	zicheng xu
procedure	过程	guocheng
function	函数	hanshu
parameter	参数	canshu
argument	实参	shican

return value 返回值 fanhuizhi

local variable 局部变量 jubu bianliang

global variable 全局变量 quanju bianliang

structured programming 结构化 jiegouhua

modular programming 模块化编程 mokuaihua biancheng

comment 注释 zhushi

pseudocode 伪代码 weidaima

flowchart 流程图 liuchengtu

trace table 追踪表 zhuizong biao

--- TOTAL TERMS IN GLOSSARY: 165 ---

11. PSEUDOCODE REFERENCE CARD (Cambridge Standard)

VARIABLES & ASSIGNMENT

DECLARE <variable_name> : <data_type>

<variable_name> ← <value>

Data types: INTEGER, REAL, BOOLEAN, STRING, CHAR, DATE

Example:

DECLARE Score : INTEGER

Score ← 85

OUTPUT

OUTPUT <message_or_value>

Example:

OUTPUT "Hello, World!"

OUTPUT "Your score is: ", Score

INPUT

INPUT <variable_name>

Example:

INPUT UserName

OUTPUT "Hello, ", UserName

SELECTION

IF <condition> THEN

<statements>

ELSE

<statements>

ENDIF

CASE OF <expression>

<value_1> : <statements>

<value_2> : <statements>

OTHERWISE : <statements>

ENDCASE

Example:

IF Score >= 50 THEN

OUTPUT "Pass"

ELSE

OUTPUT "Fail"

ENDIF

ITERATION

FOR <counter> ← <start> TO <end>

<statements>

NEXT <counter>

FOR <counter> ← <start> TO <end> STEP <increment>

<statements>

NEXT <counter>

REPEAT

<statements>

UNTIL <condition>

WHILE <condition> DO

<statements>

ENDWHILE

Example (count-controlled):

FOR i ← 1 TO 5

OUTPUT i

NEXT i

Example (condition-controlled):

WHILE Score <> -1 DO

INPUT Score

ENDWHILE

ARRAYS

DECLARE <array_name> : ARRAY[<lower>:<upper>] OF <data_type>

Example:

DECLARE Names : ARRAY[1:10] OF STRING

Names[1] ← "Alice"

OUTPUT Names[1]

2D array:

DECLARE Grid : ARRAY[1:5, 1:5] OF INTEGER

STRING HANDLING

LENGTH(<string>) → returns integer length

LEFT(<string>, <n>) → returns first n characters

RIGHT(<string>, <n>) → returns last n characters

MID(<string>, <pos>, <n>) → returns n characters from position

TO_UPPER(<string>) → converts to uppercase

TO_LOWER(<string>) → converts to lowercase

NUM_TO_STR(<number>) → converts number to string

STR_TO_NUM(<string>) → converts string to number

ASC(<character>) → returns ASCII code

CHR(<integer>) → returns character from ASCII code

FILE HANDLING

OPENFILE <file_name> FOR READ / WRITE / APPEND

READFILE <file_name>, <variable>

WRITEFILE <file_name>, <data>

CLOSEFILE <file_name>

EOF(<file_name>) → returns TRUE at end of file

Example:

```
OPENFILE "Scores.txt" FOR READ
WHILE NOT EOF("Scores.txt") DO
  READFILE "Scores.txt", Line
  OUTPUT Line
ENDWHILE
CLOSEFILE "Scores.txt"
```

SUBROUTINES

PROCEDURE <name>(<parameters>)

<statements>

ENDPROCEDURE

FUNCTION <name>(<parameters>) RETURNS <data_type>

<statements>

RETURN <value>

ENDFUNCTION

CALL <procedure_name>(<arguments>)

Example:

```
FUNCTION Add(Num1 : INTEGER, Num2 : INTEGER) RETURNS INTEGER
  RETURN Num1 + Num2
ENDFUNCTION
Result ← Add(5, 3)
OUTPUT Result       // Outputs 8
```

BUBBLE SORT (Cambridge standard pseudocode)

```
PROCEDURE BubbleSort(Data : ARRAY[1:n] OF INTEGER)
  DECLARE Temp : INTEGER
  DECLARE Swapped : BOOLEAN
  REPEAT
    Swapped ← FALSE
    FOR i ← 1 TO n - 1
      IF Data[i] > Data[i + 1] THEN
        Temp ← Data[i]
        Data[i] ← Data[i + 1]
        Data[i + 1] ← Temp
        Swapped ← TRUE
      ENDIF
    NEXT i
  UNTIL Swapped = FALSE
ENDPROCEDURE
```

RANDOM NUMBERS

RANDOM_INT(<min>, <max>) → returns random integer in range

RANDOM() → returns random real 0.0 to 1.0

LOGICAL / RELATIONAL OPERATORS

= equal to <> not equal to

> greater than < less than

>= greater than or equal <= less than or equal

AND logical AND OR logical OR

NOT logical NOT

ARITHMETIC OPERATORS

+ addition - subtraction

* multiplication / division (real result)

DIV integer division MOD modulus (remainder)

^ exponent / power

12. SQL SYNTAX REFERENCE CARD

BASIC QUERY COMMANDS

```
SELECT <column_name(s)>
FROM <table_name>
WHERE <condition>
ORDER BY <column_name> ASC | DESC;
-- Select all columns:
SELECT * FROM <table_name>;
-- Select specific columns:
SELECT FirstName, LastName FROM Students;
-- Select with condition:
SELECT * FROM Students WHERE Age > 16;
-- Select with multiple conditions:
SELECT * FROM Students WHERE Age > 16 AND Grade = 'A';
-- Sort results:
SELECT * FROM Students ORDER BY LastName ASC;
```

WILDCARDS

```
%   represents zero or more characters
_   represents a single character
Examples:
SELECT * FROM Students WHERE FirstName LIKE 'J%';     -- Names starting with J
SELECT * FROM Students WHERE LastName LIKE '_mith';    -- e.g. Smith
SELECT * FROM Products WHERE Name LIKE '%computer%';   -- Contains 'computer'
```

INSERT, UPDATE, DELETE

```
-- Add a new record:
INSERT INTO <table_name> (column1, column2, column3)
VALUES (value1, value2, value3);
Example:
INSERT INTO Students (StudentID, FirstName, LastName, Age)
VALUES (101, 'Li', 'Wei', 16);
-- Modify existing records:
UPDATE <table_name>
SET column1 = value1, column2 = value2
WHERE <condition>;
```

Example:

UPDATE Students SET Age = 17 WHERE StudentID = 101;

-- Remove records:

DELETE FROM <table_name> WHERE <condition>;

Example:

DELETE FROM Students WHERE StudentID = 101;

WARNING: Without WHERE, UPDATE and DELETE affect ALL records!

JOIN OPERATIONS

-- Inner join: returns only matching records:

SELECT *

FROM TableA

INNER JOIN TableB ON TableA.ForeignKey = TableB.PrimaryKey;

Example:

SELECT Students.FirstName, Enrolments.CourseName

FROM Students

INNER JOIN Enrolments ON Students.StudentID = Enrolments.StudentID;

AGGREGATE FUNCTIONS

COUNT(*) → counts number of rows

SUM(<column>) → total of numeric column

AVG(<column>) → average of numeric column

MAX(<column>) → maximum value

MIN(<column>) → minimum value

Example:

SELECT COUNT(*) FROM Students; -- How many students?

SELECT AVG(Age) FROM Students; -- Average age

SELECT MAX(Score) FROM Exams; -- Highest score

DATA DEFINITION (advanced)

CREATE TABLE <table_name> (

<column_name> <data_type> <constraints>,

<column_name> <data_type> <constraints>,

PRIMARY KEY (<column_name>)

);

Common data types: INTEGER, REAL, TEXT, DATE, BOOLEAN

Constraints: PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE

Example:

CREATE TABLE Students (

```
  StudentID   INTEGER     PRIMARY KEY,
  FirstName   TEXT        NOT NULL,
  LastName    TEXT        NOT NULL,
  Age         INTEGER,
  Email       TEXT        UNIQUE
);
```

13. BOOLEAN LOGIC REFERENCE CARD

LOGIC GATE SYMBOLS & TRUTH TABLES

The six logic gates required by CIE 0478:

```
┌──────────────┬──────────────────────────────────────────────────────────────┐
  Gate         → Description & Truth Table
  AND          → Output TRUE only if BOTH inputs are TRUE
  A ──┐        → A   B   Output
  AND →       0   0    0
  B ──┘        → 0   1    0
                 1   0    0
  OR           → Output TRUE if EITHER or BOTH inputs are TRUE
  A ──┐        → A   B   Output
  OR  →       0   0    0
  B ──┘        → 0   1    1
                 1   0    1
  NOT          → Output is the OPPOSITE of the single input
  A ──NOT──    → A   Output
                 o → 0    1
                 1    0
  NAND         → Output is NOT AND — opposite of AND
  (AND gate followed by NOT circle)
  A ──┐o       → A   B   Output
  NAND →      0   0    1
  B ──┘        → 0   1    1
                 1   0    1
                 1   1    0
  NOR          → Output is NOT OR — opposite of OR
  (OR gate followed by NOT circle)
  A ──┐o       → A   B   Output
  NOR  →      0   0    1
  B ──┘        → 0   1    0
```

1 0 0

1 1 0

XOR → Output TRUE if inputs are DIFFERENT

(Exclusive OR — "one or the other but not both")

A —⊕— → A B Output

XOR → 0 0 0

B —┘ → 0 1 1

1 0 1

1 1 0

LOGIC EXPRESSIONS & CIRCUITS

Writing expressions from circuits:

- Trace from inputs through each gate to the output
- Name the output of each intermediate gate if helpful

Example: A —┬— AND —┬— OR —— X

B —┘ │

C ———————┘

Expression: X = (A AND B) OR C

Drawing circuits from expressions:

1. Identify the final operation (the "outermost" operator)
2. That becomes the final gate before the output
3. Work inward, drawing gates for each sub-expression

Example: X = NOT(A AND B)

4. Final operation: NOT → NOT gate at the end
5. Inside: A AND B → AND gate feeding into NOT
6. A and B connect to AND, AND output → NOT, NOT output = X

HALF-ADDER

A half-adder adds two single binary digits:

Inputs: A, B (each is 0 or 1)

Outputs: Sum, Carry

A —┬— XOR —— Sum

│

B —┤

│

├— AND —— Carry

Truth table:

A B Sum Carry

Logic expressions:

Sum = A XOR B

Carry = A AND B

Note: A full adder (extension) adds three bits including a carry-in.

SUMMARY OF GATE SYMBOLS (CIE EXAM STYLE)

AND gate: D-shaped, flat back

OR gate: Curved front, pointed back

NOT gate: Triangle with circle (○) on output

NAND gate: AND shape with circle on output

NOR gate: OR shape with circle on output

XOR gate: OR shape with extra curved line on input side

Remember: The circle (○) always means NOT/inversion.

END OF COURSE OVERVIEW — Year 10 Semester 1

Victoria Park Academy Shenzhen — CIE IGCSE Computer Science (0478)